The Hong Kong Polytechnic University

Department of Computing

COMP4913 Capstone Project
Report (Interim)

Financial Time-Series Prediction Using Advanced Neural Network Models

Student Name:	CHAN Cheung Hong
Student ID No.:	22081328D
Programme-Stream Code:	61435-FCS
Supervisor:	Dr Yujie Wu
Co-Examiner:	
Submission Date:	07 January 2026

Abstract

Forecasting financial time series is difficult due to noise, non-stationarity, and regime changes [13][11]. This project studies whether advanced neural sequence models can improve short-horizon prediction compared with classical baselines under a leakage-free evaluation [5][1][13]. Using public daily market data from Yahoo Finance for SPY (core) and BTC-USD (extension) from 2010-01-01 onward, the target is the next-day log return. Models are compared using a chronological 70/15/15 train/validation/test split, with feature standardization fitted on the training set only. Performance is measured by mean absolute error (MAE), root mean squared error (RMSE), and directional accuracy (DA).Preliminary experiments on SPY using a lag window length L = 10 show that ridge regression substantially improves over a persistence baseline (RMSE decreases from 0.0144 to 0.00993; MAE decreases from 0.00953 to 0.00664; DA is about 0.517). A compact LSTM achieves similar error (RMSE 0.00983; MAE 0.00658) while DA remains around 0.516, suggesting that reducing magnitude error is easier than improving direction. Next steps include implementing a capacity-matched Transformer, expanding feature sets (e.g., volume/technical indicators), running robustness checks across horizons and periods, and extending evaluation to BTC-USD.

Keywords: SPY, BTC-USD, time-series forecasting, LSTM, Transformer, baselines.

Table of Contents

Abstract **2**
Table of Contents **3**
List of Tables and Figures **4**
1. Introduction **5**
1.2 Problem statement 5
1.3 Objectives and success criteria 6
1.4 Scope and non-goals 6
2. Related Work **8**
3. Data and Preprocessing **9**
3.1 Data sources and period 9
3.2 Target definition (log-return) 9
3.3 Features (log price, log1p(volume), optional technical indicators) 9
3.4 Train/validation/test split (chronological) and no-leakage checklist 9
3.5 Robustness evaluation (planned) 10
4. Methodology **11**
4.1 Baselines 11
4.2 LSTM model (core) 11
4.3 Transformer model (stretch) 11
4.4 Evaluation metrics 12
5. Experimental Setup **12**
5.1 Implementation details 12
5.2 Training protocol 12
5.3 Hyperparameter selection 12
5.4 No-leakage checklist (enforced) 13
5.5 Planned ablations 13
6. Progress to Date and Preliminary Results **14**
6.1 Implemented pipeline and reproducibility artifacts 14
6.2 Summary results (SPY next-day return prediction) 14
6.3 Figures 14
6.4 LSTM preliminary run (if available) and observations 16
7. Project Plan (Updated) **17**
7.1 Work completed vs. original plan 17
7.2 Updated milestones to final submission 17
7.3 Risks and mitigation 17
8. Ethics, Academic Integrity, and GenAI Use **19**
9. Conclusion and Next Steps **20**
References **21**
Appendix A: Reproducibility (Commands) **22**

List of Tables and Figures

Figure 1: LSTM true vs predicted next-day log returns (test slice)

Figure 2: LSTM scatter plot (true vs predicted)

Figure 3: LSTM learning curve (training and validation loss)

1. Introduction

1.1 Background and motivation

Financial markets are influenced by macroeconomic conditions, market microstructure, and the behaviour of market participants, producing time series that are volatile, non-stationary, and subject to regime shifts [13]. Accurate forecasting of such series is valuable for risk management, investment decision support, and automated allocation [13]. Traditional statistical approaches (e.g., linear regression and ARIMA) are logically simple and efficient, but their assumptions and limited ability to model nonlinear dynamics often break under changing market regimes and structural shifts.

Deep learning offers an alternative by learning nonlinear temporal patterns directly from data. Recurrent architectures (RNNs/LSTMs) are widely used because their gated structure can capture multi-lag dependencies [5][12]. Feed-forward forecasting models such as N-BEATS demonstrate strong sequence modelling capability without recurrence [8][9]. More recently, Transformers have attracted attention for modelling long-range dependencies with parallel training and flexible receptive fields, and domain-specific variants (e.g., Hidformer) report promising performance in equity forecasting [1][2][6]. These developments suggest that neural sequence models may improve predictive accuracy and robustness—if evaluated rigorously under leakage-free protocols [7].

1.2 Problem statement

This project investigates whether advanced neural architectures—specifically LSTM and Transformer models—can produce accurate and robust forecasts of financial time series using only public daily market data. The study focuses on both short-horizon forecasting (1-day ahead, core task) and longer-horizon forecasting (10-day ahead, stretch task), examines the effect of including volume alongside price, and evaluates model performance using both statistical error metrics (MAE, RMSE) and directional measures (Directional Accuracy, DA). Particular emphasis is placed on avoiding common evaluation pitfalls such as data leakage, narrow test windows, and incomplete metric reporting.

Multi-year daily data for SPY (S&P 500 ETF) and BTC-USD (Bitcoin spot), collected from Yahoo Finance, serve as the primary empirical basis. Data are preprocessed via windowing, normalization, and chronological train/validation/test splits. The work follows a Core + Stretch design to ensure feasibility and clear attribution:

- **Core:** SPY 1-day-ahead forecasting using classical baselines and LSTM, under leakage-free evaluation.

- **Stretch:** BTC-USD and/or 10-day horizon forecasting, plus a capacity-matched Transformer for direct comparison.

1.3 Objectives and success criteria

Aim. Design, implement, and evaluate neural architectures for forecasting financial time series, focusing on LSTM and Transformer models for price-only and price+volume inputs across short and longer horizons.

Objectives.

- **O1 Data pipeline:** Reproducible Yahoo Finance pipeline for SPY and BTC-USD: retrieve daily close + volume, clean, window, normalize, and deterministically split into train/validation/test.

- **O2 Baselines:** Implement persistence and linear/ridge regression baselines under identical data and evaluation protocols.

- **O3 LSTM model:** Implement and tune LSTM for price-only and price+volume, with saved checkpoints and learning curves.

- **O4 Transformer model:** Implement a capacity-matched Transformer encoder for fair comparison with LSTM.

- **O5 Evaluation:** Evaluate with MAE, RMSE, and DA; include robustness checks (rolling evaluation) and optional statistical tests (e.g., Diebold–Mariano).

- **O6 Ablation & sensitivity:** Study impacts of sequence length, learning rate, and input type (price vs price+volume), as well as model depth.

- **O7 Reproducibility:** Provide a clean PyTorch codebase with fixed seeds, configuration files, and scripts that regenerate tables/figures.

Success criteria. The interim milestone is considered successful if it demonstrates (i) a feasible plan to final submission, (ii) a leakage-free evaluation framework, and (iii) measurable improvements over baselines supported by reproducible artifacts (code, metrics, figures).

1.4 Scope and non-goals

Scope: Daily OHLCV-based forecasting for SPY and BTC-USD, focusing on next-day and (optionally) 10-day horizon returns; comparison between baselines, LSTM, and Transformer under a consistent evaluation protocol.

Non-goals: Intraday prediction, news/alternative data integration, and full trading strategy backtesting are out of scope for this project and reserved for future work.

2. Related Work

Classical forecasting methods such as linear regression and ARIMA have long been used for time series prediction, but they are limited in modelling nonlinear dependencies and are sensitive to regime changes typical in financial markets. Deep learning approaches mitigate some limitations by learning representations directly from data. LSTM-based architectures are widely applied in financial forecasting due to their ability to model longer dependencies through gating mechanisms. Probabilistic autoregressive recurrent models (e.g., DeepAR) introduce additional temporal structure through likelihood-based training [12][4].

Transformers remove recurrence and can capture long-range context using attention, enabling parallel training and flexible receptive fields. A growing body of work explores Transformer variants for time series forecasting and questions when and why they outperform recurrent models. Domain-specific Transformer variants (e.g., Hidformer) have been reported to perform strongly on equity forecasting tasks. Recent architectures such as TimesNet further highlight the rapid evolution of forecasting paradigms by modelling time series with additional structural inductive biases [10][3].

Despite progress, three practical issues frequently limit deployment: (1) effectively integrating additional signals (e.g., volume/liquidity) in price prediction to improve directional accuracy; (2) forecast instability as horizon increases due to compounding uncertainty and concept drift; and (3) evaluation bias caused by leakage or narrow test windows, as well as insufficient metrics (often reporting only RMSE/MAE while ignoring decision-relevant directional or risk-aware metrics). This project addresses these gaps by comparing LSTM and Transformer models under a leakage-free evaluation and by reporting both statistical and directional metrics [11].

3. Data and Preprocessing

3.1 Data sources and period

Daily market data are downloaded from Yahoo Finance using the `yfinance` library. Assets include:

- **SPY** (S&P 500 ETF) as the primary dataset (core).
- **BTC-USD** (Bitcoin spot) as an extension dataset (stretch).

The analysis uses a multi-year period (final report will specify exact start/end dates used in experiments). Data fields include daily adjusted close (or close with auto-adjust) and volume.

3.2 Target definition (log-return)

Let C_t be the adjusted closing price on day t. The one-step log return on day t is defined as:

$r_t = \log(C_t) - \log(C_{t-1})$.

For forecasting, the core task predicts the next-day log return r_{t+1} using information available up to day t. A stretch task considers a 10-day horizon, predicting r_{t+10}. Log returns are used because they are scale-invariant and commonly used in financial modelling [13].

3.3 Features (log price, log1p(volume), optional technical indicators)

Input features include:

- **Price-only:** log price or log returns.
- **Price + volume:** log price or log returns plus log1p(volume), where log1p(volume) means log(1 + volume). Optional extensions for ablation include simple technical indicators (e.g., moving average, rolling volatility), but the core design remains lightweight.

3.4 Train/validation/test split (chronological) and no-leakage checklist

For current preliminary results, inputs are constructed using a lag window length L = 10 (the ten most recent returns). Samples are split chronologically into 70% training, 15% validation, and 15% test. Feature standardization is fitted only on the training set, then applied to validation and test sets. No shuffling is performed. These rules ensure leakage-free evaluation.

Planned extension: experiments will also test longer windows such as L = 30 and L = 60 and will add rolling-window evaluation.

3.5 Robustness evaluation (planned)

To test stability under regime shifts, rolling-window evaluation and/or multiple testing windows will be used in later stages. This provides evidence of generalization beyond a single fixed split.

4. Methodology

4.1 Baselines

Two primary baselines are used for fair comparison under identical preprocessing and evaluation:

- **Naïve persistence:** predict the next-day return equals today's return.
- **Persistence baseline:** predict $r_{t+1} = r_t$.
- **Ridge regression:** trained on lagged features (e.g., last 10 returns or lagged log prices), with hyperparameter selection via validation.

Optional sanity checks include seasonal-naïve or ARIMA where appropriate.

4.2 LSTM model (core)

The LSTM model is implemented in PyTorch as a compact forecaster:

- 1–2 LSTM layers
- hidden size 64–128
- dropout 0.1–0.3
- final linear layer for prediction

Two input variants are evaluated:

- Price-only
- price + volume

4.3 Transformer model (stretch)

A Transformer encoder is implemented for sequence forecasting:

- 2–3 encoder blocks
- 4–8 attention heads
- model dimension 64–128
- feed-forward dimension 2–4× model dimension
- positional encoding (e.g., sinusoidal)
- dropout 0.1–0.3

Capacity matching: The Transformer configuration is chosen to be comparable in parameter scale to the LSTM to ensure a fair comparison rather than simply increasing model size.

4.4 Evaluation metrics

Models are evaluated using:

- **MAE**
- **RMSE**
- **Directional Accuracy (DA)**: fraction of times predicted and actual return share the same sign.

In later stages, statistical tests (paired t-test or Diebold–Mariano) may be applied to compare error series across models [14].

5. Experimental Setup

5.1 Implementation details

- Language: Python (target 3.11)
- Libraries: PyTorch, NumPy, pandas, scikit-learn, Matplotlib, yfinance
- Reproducibility: version-controlled codebase, fixed random seeds, config files (JSON/YAML), and scripts that output all tables/figures.

5.2 Training protocol

- Optimizer: Adam ($\beta1=0.9$, $\beta2=0.999$)
- Learning rate: 0.001 (with step or cosine decay as optional extension)
- Batch size: 32–128
- Max epochs: up to 100
- Early stopping: patience ≈ 10 based on validation RMSE (or MSE)

The best checkpoint is saved using the validation metric.

5.3 Hyperparameter selection

Hyperparameters (e.g., sequence length 10, hidden size, dropout, learning rate) are selected using the validation set. The selection protocol is consistent across baselines and deep models to ensure comparability.

5.4 No-leakage checklist (enforced)

- Chronological splits (no shuffling)
- Scalers fitted on training data only
- Inputs use information up to time t only; targets are at time t plus horizon h
- Any tuning uses validation set only; test set is used once for final reporting

5.5 Planned ablations

Sensitivity studies will vary:

- sequence length (e.g., 20/30/60)
- learning rate (0.01/0.001/0.0005)
- input type (price-only vs price+volume)
- model depth (LSTM layers; Transformer blocks)

6. Progress to Date and Preliminary Results

6.1 Implemented pipeline and reproducibility artifacts

A reproducible baseline pipeline was implemented to download SPY data using yfinance, compute daily log returns, construct lagged inputs, and evaluate models under a leakage-free chronological split. All experiments are version-controlled in Git. The pipeline outputs metrics in JSON and CSV format, and diagnostic plots including test-slice trajectories and scatter plots. The baseline code and results are committed, and the environment dependencies are recorded in requirements.txt to support replication on different machines.

6.2 Summary results (SPY next-day return prediction)

Model	MAE	RMSE	Directional Accuracy (DA)
persistence	0.009526	0.014395	0.5174
ridge	0.006639	0.009929	0.5174
lstm	0.006582	0.009827	0.5158

Note: Ridge used alpha=100.0 chosen by validation RMSE.

6.3 Figures

Figure 1: LSTM true vs predicted next-day log returns (test slice).

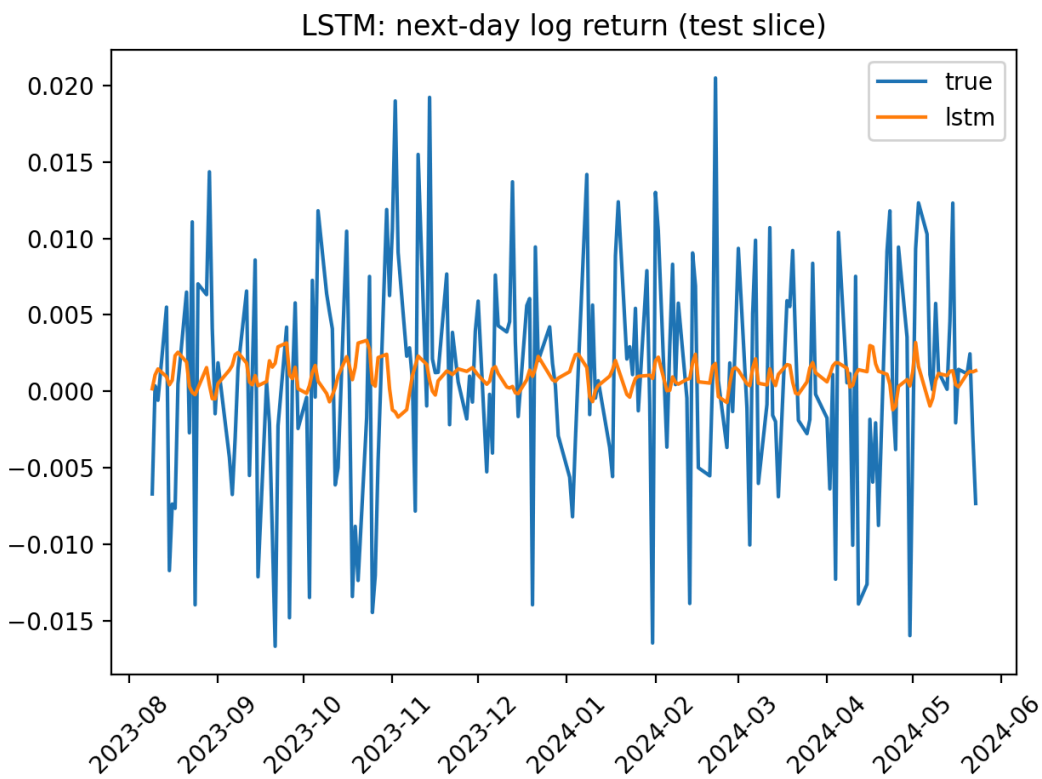


Figure 2: LSTM scatter plot (true vs predicted next-day log returns).

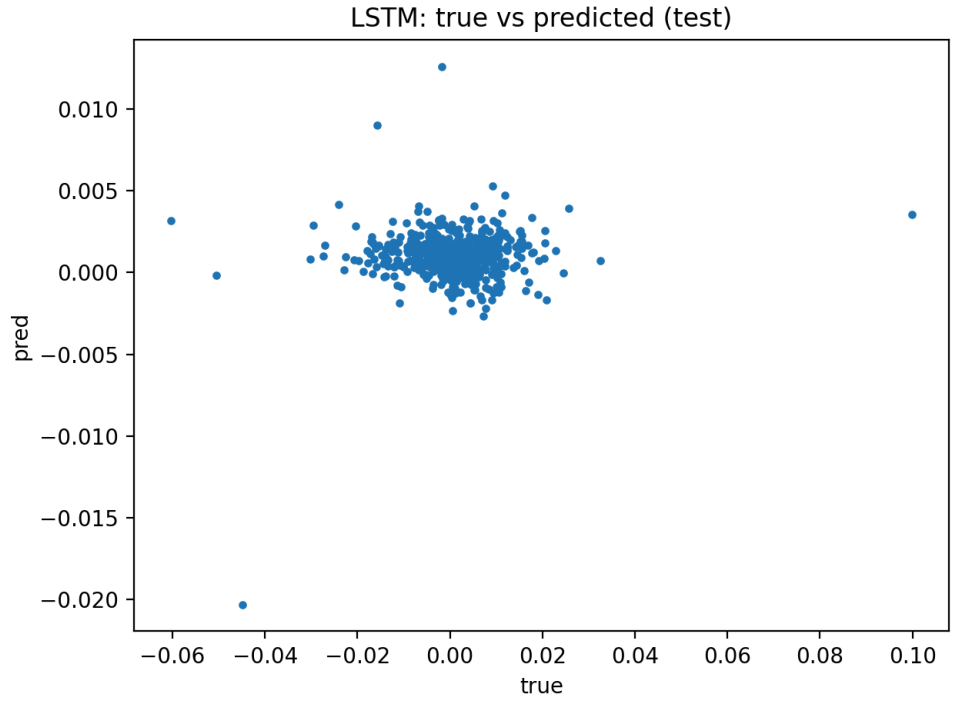
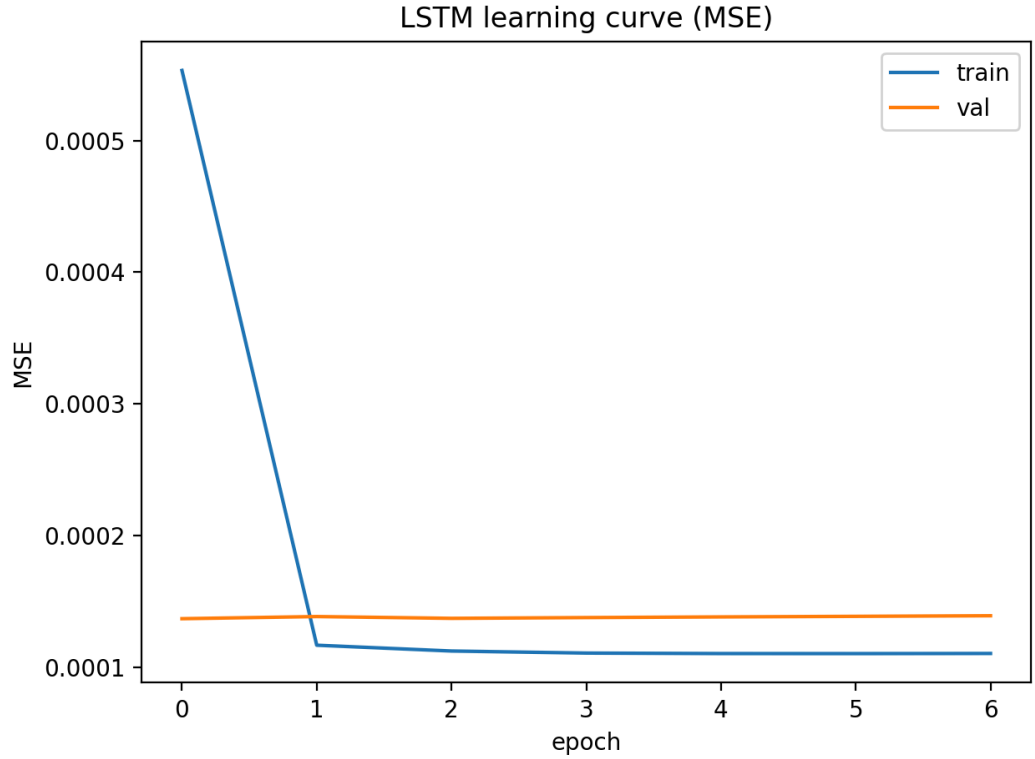


Figure 3: LSTM learning curve (training and validation loss).



6.4 LSTM preliminary run (if available) and observations

A compact LSTM was trained for next-day SPY log return prediction using a sequence length of 10, one LSTM layer with hidden size 64, batch size 64, learning rate 0.001, and early stopping based on validation loss. On the test set, the LSTM achieved MAE = 0.006582, RMSE = 0.009827, and DA = 0.5158.

Compared with ridge regression (RMSE = 0.009929 and MAE = 0.006639), the LSTM provides a marginal improvement in error magnitude, while directional accuracy remains close to about 0.516 to 0.517. This suggests that reducing prediction error is achievable with the current pipeline, but improving directional accuracy may require richer inputs such as volume or technical indicators, robustness checks across different periods, and careful regularization to avoid overfitting.

7. Project Plan (Updated)

7.1 Work completed vs. original plan

The project is behind the originally proposed schedule. Completed work includes a leakage-free pipeline for SPY, implementation of persistence and ridge baselines with saved metrics and plots, and an initial LSTM run with early stopping and recorded test metrics. Remaining work includes implementing the Transformer baseline, adding volume as an input, testing longer horizons (10-day), running robustness evaluation such as rolling windows, performing ablations, and completing the final report.

7.2 Updated milestones to final submission

Jan 8 to Jan 15, 2026: clean repository, unify scripts, generate one combined results table for persistence, ridge, and LSTM.

Jan 16 to Jan 31, 2026: add price plus volume input, tune LSTM (sequence length, dropout), keep learning curves and checkpoints.

Feb 1 to Feb 20, 2026: implement a capacity-matched Transformer and run SPY comparisons under identical settings.

Feb 21 to Mar 10, 2026: robustness evaluation using rolling windows and different time periods, plus 10-day horizon experiments.

Mar 11 to Mar 25, 2026: ablations for sequence length, learning rate, model depth, and input type.

Mar 26 to Apr 10, 2026: final analysis, writing, proofreading, and final presentation preparation.

7.3 Risks and mitigation

Regime shift and concept drift: use rolling-window evaluation and compare across multiple periods.

Overfitting in deep models: use early stopping, dropout, capacity matching, and conservative hyperparameter search.

Compute constraints: use compact models and Colab GPU only when needed.

Leakage risk: enforce strict chronological splits and train-only scaling, and review each experiment with a no-leakage checklist.

8. Ethics, Academic Integrity, and GenAI Use

Generative AI tools were used for writing assistance such as outlining and grammar improvements, and for code scaffolding suggestions. All experimental code was implemented and executed by the student, and all reported results and figures were produced from the student's own runs. The student reviewed and verified the correctness of the code and ensured leakage-free evaluation procedures. Only public data from Yahoo Finance were used.

9. Conclusion and Next Steps

This interim report established a leakage-free experimental pipeline for SPY next-day log return prediction and produced preliminary results for persistence, ridge regression, and a compact LSTM. Ridge regression shows a substantial improvement over persistence in MAE and RMSE, while the LSTM provides only a marginal improvement over ridge and does not yet improve directional accuracy. Next steps are to add volume and optional technical indicators, implement a capacity-matched Transformer under the same protocols, extend experiments to the 10-day horizon and BTC-USD, and conduct robustness evaluation and ablation studies to identify when deep models provide consistent benefits.

References

1. Lim, B., Arik, S. O., Loeff, N., & Pfister, T. (2021). *Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting.* International Journal of Forecasting, 37(4), 1748–1764.
   https://doi.org/10.1016/j.ijforecast.2021.03.012
2. Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., & Zhang, W. (2021). *Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting.* AAAI Conference on Artificial Intelligence (AAAI 2021).
   https://ojs.aaai.org/index.php/AAAI/article/view/17325
3. Nie, Y., Liu, S., Zhang, S., & Luo, P. (2022). *Long-term Forecasting with Transformers: A Survey and Benchmark.* arXiv preprint arXiv:2211.14730.
   https://arxiv.org/abs/2211.14730
4. Seabe, P. L., Moutsinga, C. R. B., & Pindza, E. (2023). *Forecasting Cryptocurrency Prices Using LSTM, GRU, and Bi-Directional LSTM: A Deep Learning Approach. Fractal and Fractional,* 7(2), Article 203.
   https://doi.org/10.3390/fractalfract7020203
5. Fjellström, C. (2022). *Long Short-Term Memory Neural Network for Financial Time Series.* arXiv preprint arXiv:2201.08218.
   https://arxiv.org/abs/2201.08218
6. Szydłowski, K. L., & Chudziak, J. A. (2024). *Hidformer: Transformer-style Neural Network in Stock Price Forecasting.* arXiv preprint arXiv:2412.19932.
   https://arxiv.org/abs/2412.19932
7. Zeng, A., Liu, X., Xu, Y., & Liu, Q. (2023). *Are Transformers Effective for Time Series Forecasting?* arXiv preprint arXiv:2301.10766.
   https://arxiv.org/abs/2205.13504
8. Oreshkin, B., Carpov, D., Chapados, N., & Bengio, Y. (2019). *N-BEATS: Neural Basis Expansion Analysis for Interpretable Time Series Forecasting.* NeurIPS 2019.
   https://arxiv.org/abs/1905.10437
9. Challu, C., Tay, F., Rangapuram, S. S., Salinas, D., Flunkert, V., & Gasthaus, J. (2022). *N-HiTS: Neural Hierarchical Interpolation for Long-term Time Series Forecasting.* arXiv preprint arXiv:2201.12886.
   https://arxiv.org/abs/2201.12886

10. Wu, H., Xu, J., Wang, J., & Long, M. (2022). *TimesNet: Temporal 2D-variation Modeling for General Time Series Analysis.* International Conference on Learning Representations (ICLR 2023).
https://arxiv.org/abs/2210.02186
11. Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2020). *The M4 Competition: Results, Findings, and Implications.* International Journal of Forecasting, 36(1), 1–46.
https://doi.org/10.1016/j.ijforecast.2019.04.014
12. Salinas, D., Flunkert, V., Gasthaus, J., & Januschowski, T. (2020). *DeepAR: Probabilistic Forecasting with Autoregressive Recurrent Networks. International Journal of Forecasting,* 36(3), 1181-1191.
https://doi.org/10.1016/j.ijforecast.2019.07.001
13. López de Prado, M. L. (2018). *Advances in Financial Machine Learning.* Wiley.
https://www.wiley.com/en-us/Advances+in+Financial+Machine+Learning-p-9781119482086
14. Diebold, F. X., & Mariano, R. S. (1995). *Comparing Predictive Accuracy.* Journal of Business & Economic Statistics, 13(3), 253–263.
https://doi.org/10.1080/07350015.1995.10524599

Appendix A: Reproducibility (Commands)

```
python -m venv .venv

pip install -r requirements.txt

python src/day1_baselines.py

python src/day2_lstm.py
```